List Part #2

Monday, March 2, 2026 9:26 AM

List #2

Recap:

- List syntax: [item1, item2, item3]
- index: การเข้าไปเอาข้อมูล ตาม index /ตำแหน่ง
  ↳ index เริ่มที่ 0 list[index]
- update: list[index] = new_value

List Operators

- \+ : เอา list มาต่อกัน ได้ list ใหม่

  [1, 2, 3] + [2, 3] → [1, 2, 3, 3, 3]

- * → syntax: list * n
  ↳ เอา list ต่อตัวเอง n ครั้ง
  [1, 2, 3] * 3 → [1, 2, 3, 1, 2, 3, 1, 2, 3]

- in → ใช้เช็คว่า ของอยู่ใน list ไหม?
  ↳ True หรือ False

e.g. 3 in [1, 2, 3] → True

4 in [1, 2, 3] → False

ex เช็ค ID จาก user

```
IDs = [1234, 4367, 9321, 7723]
user_id = int(input("User ID: "))
if user_id in IDs:
    print("Found")
```

```
else:
    print("Not Found")
```

List Methods

- คำสั่ง เฉพาะ list
- Syntax   list.method_name()

ขึ้นอยู่กับ method

1) append(item) method

↳ Syntax: list.append(item)

↳ เพิ่ม item เข้าไปด้านหลังของ list

[1, 2, 3].append(5) → [1, 2, 3, 5]

* เพิ่มใน list นั้นเลยไม่ได้ list ใหม่ออกมา
เหมือน +

e.x1

```
my_list = [1,2,3]
my_list.append(5)      ← # my_list += [5]
print(my_list)         → [1, 2, 3, 5]
my_list.append(7)
my_list.append(9)
print(my_list)         → [1,2,3,5,7,9]
```

e.x2 สร้าง list ด้วย append()

```
my_list = [3, 5, 7, 9]
new_list = [ ]     ← empty
for item in my_list:
    new_list.append(item)
```

#1 .append(3)
#2 .append(5)
#3 .append(7)
#4 .append(9)

```
new_list.append(item)
print(new_list) → [3, 5, 7, 9]
```

#4 .append(9)

e.x.3 สร้าง list ใหม่จาก user โดยใช้ append

1) รับ n มาก่อนว่า จะมีกี่ item
2) loop รับ อีก n ค่า มา append ใน list

```
n = int(input("n: "))
my_list = []
for i in range(n):
    item = input("item: ")
    my_list.append(item)
print(my_list)
```

user พิมพ์
n: 3
item: 2
item: 3
item: 4
['2', '3', '4']

• insert(index, item) method

↳ เพิ่ม (แทรก/) ค่า item ไปที่ตำแหน่ง index แล้วผลัก ค่า ที่อยู่ ที่ index และหลังจากนั้นไปหนึ่งลำดับ

0 1 2
[1, 2, 3].insert(1, 55)

[1, 55, 2, 3]

* เปลี่ยน list นั้น ทันที

remove(item) method

• ลบ item ตัวที่ ตัวแรกสุด ออกจาก list
* ถ้า item ไม่มีอยู่ใน list จะ error!

[10, 20, 30, 20].remove(20)

[10, ~~20~~, 30, 20].remove(20)

↓

[10, 30, 20]

[1, 2, 3].remove(5) → error!

เพราะ 5 ไม่อยู่ใน list

ต้องเช็คก่อนว่า item อยู่ใน list

↳ เช็ค item in list

---

del function

Syntax: del list[i]

↳ ลบของตาม index

ex.
```
seq = [11, 22, 33, 44]
del seq[1]
print(seq)  → [11, 33, 44]
del seq[1]
print(seq)  → [11, 44]
```

.sort() method

↳ จะเรียงของใน list จากน้อยไปมาก

[5, 7, 2, 9].sort()

เรียง

↓

[2, 5, 7, 9]

* เปลี่ยน list นั้นเลย

e.g. my_list = [5, 7, 2, 9]

↑ เปลี่ยน list นั้นเลย

e.g. my_list = [5, 7, 2, 9]
my_list.sort()
print(my_list) → [2, 5, 7, 9]

---

sorted(list) function

↳ คือ เรียง list แต่จะได้ list ใหม่ ออกมา

my_list = [5, 7, 2, 9]
new_list = sorted(my_list)
print(my_list) → [5, 7, 2, 9]
print(new_list) → [2, 5, 7, 9]

* sorted(list, ascend = True)

True = จากน้อยไปมาก
False = มากไปน้อย

Useful built-in functions

- sum(list)

↳ จะส่งค่า ผลรวม ของ list ออกมา

e.g. total = sum([1, 2, 3, 4]) = 10
print(total) → $\boxed{10}$

e.g.2 scores = [10, 20, 30, 40]
mean = sum(scores) / len(scores)

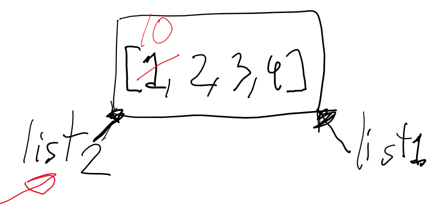
- max(list) → จะส่งคืนค่า สูงสุดใน list

- min(list) → จะส่งคืนค่า ต่ำสุดใน list

e.g. my_list = [10, 5, 2, 9]
print(max(my_list), min(my_list))
10 2

10 2

List Referencing



* = จะไม่ copy / สร้าง list ใหม่

e.g.

```
list1 = [10, 2, 3, 4]
list2 = list1
list1[0] = 10
print(list1, list2)
```

ไม่ได้สร้าง list ใหม่ แล้วเก็บใน list2 แต่ list1 และ list2 ชี้ที่ list ตัวเดียวกันใน memory

ไม่ copy

[10, 2, 3, 4] [10, 2, 3, 4]

Copy list #1: ใช้ append

```
list1 = [1, 2, 3, 4]
list2 = []
for item in list1:
    list2.append(item)
```

Copy list #2: ใช้ + → ได้ list ใหม่!

```
list1 = [1, 2, 3, 4]
list2 = [] + list1
```

2D list

↳ list ซ้อน list

↳ item ใน list เป็นอะไรก็ได้

↳ item ใน list เป็นอะไรก็ได้

```
my_list = [ □ , △ ]
              0    1
print(my_list[0]) → □
```

```
my_list = [ [1, 2] , [3, 4] ]
               0        1
print(my_list[0]) → [1, 2]
print(my_list[1]) → [3, 4]
print(my_list[0][0]) → 1
         [1, 2]
print(my_list[1][1]) → 4
         [3, 4]
my_list[1].append(7)
print(my_list[1]) → [3, 4, 7]
```

2D-list as matrix

```
my_list = [ [1, 2] , [3, 4] ]
                   ↓ ก้อน
                   0          1

     0  [  [ 1 , 2 ],
             0,0  0,1
แถว  1     [ 3 , 4 ]   ]
             1,0  1,1

my_list[1][1] → 4
```

```
my_list [1][1] → 4
```

$$
\begin{array}{c|ccc}
 & 0 & 1 & 2 \\
\hline
0 & 10 & 20 & 30 \\
1 & 7 & 9 & 2 \\
2 & 6 & 3 & 4
\end{array}
$$

```
big_matrix = [ [10, 20, 30],
               [7, 9, 2],
               [6, 3, 4] ]

result = big_matrix[1][1] + big_matrix[2][0]
```

9 + 6 = 15

```
print(big_matrix[1]) → [7, 9, 2]
```

List Comprehension

↳ ตัวย่อ / short hand ของ การสร้าง list ใหม่ ด้วย append

list2 = [1, 2, 3]

ตัวเต็ม

```
list1 = []
for item in list2:
    list1.append(item)
```

ตัวย่อ

```
list1 = [item for item in list2]
```

* เปลี่ยน item, code, expression ได้

* item คือ ชื่อ var ใน loop

```
my_nums = [1, 3, 5]
squared_nums = []
```

ขอให้เป็น list!

my_nums = [1, 9, ...

```
squared_nums = []
for x in my_nums:
    squared_num.append(x**2)
```

จะได้เป็น list !

```
squared_num = [x**2 for x in my_nums]
```

List comprehension with condition

→ เอา item ที่ผ่านเงื่อนไขเป็นจริงไป append แค่นั้น

seq = [10, 20, 3, 7, 50]

* สร้าง list ใหม่จาก seq ที่มีแต่เลขคู่

```
lst = []
for x in seq:
    if x % 2 == 0:
        lst.append(x)
```

→

```
lst = [x for x in seq if x % 2 == 0]
```

* List = [item1, item2, item3]

- Operator: +, *, in
- method: .append(item), .insert(index, item)
  .remove(item), .sort()
- Useful functions: sum(), max(), min()

- Useful functions : sum( ), max( ), min( )
- 2D list
- list comprehension

Tuple คือ list ที่ update ไม่ได้ (immutable)

Syntax: tuple = (item 1, item 2, item3)

เปลี่ยนแปลงไม่ได้ คือ ใช้ พวกนี้ไม่ได้

- Update ตาม index ไม่ได้ ~~tuple[1]=10~~
- append
- insert
- remove
- sort

ที่เหลือได้หมด เช่น การเข้าถึง index

e.g.

```
           0  1  2
my_tuple = (1, 2, 3)
print(my_tuple[1])  → 2
```

~~my_tuple[1] = 5~~ ← error!